The Lustre Collective
https://thelustrecollective.com
info@thelustrecollective.com

# Lustre Find Utility (LFU) Architecture

## Table of Contents

## Revision History

| Author | Version | Date | Change |
|---|---|---|---|
| **Andreas Dilger** | 0.1 | 2026-04-03 | Initial version |
| | | | |
| | | | |
| | | | |

The Lustre Collective
https://thelustrecollective.com
info@thelustrecollective.com

## Background

Lustre filesystems regularly contain billions of files and directories, and data management requires understanding and managing these files on an ongoing basis. There are many tools that scan the filesystem namespace looking for files to process, and to gain insight into the usage and composition of the data in the filesystem. A non-exhaustive list of regular filesystem scanning tasks includes administrators looking for old files to purge, generating per-user, -group, and -project statistics to better understand usage patterns, backup utilities looking for newly-modified files to process, background data management tools looking for files to migrate between storage tiers, data integrity checking and replication, improved data compression for cold files, etc. Many of these tasks use `find` or `lfs find` while searching for files with arbitrary attributes, and/or consume Lustre Changelogs to produce an ongoing stream of recently modified files. Others depend on custom tools written to scan the filesystem.

In order to efficiently produce appropriate file lists for various tasks, it is desirable to have an efficient and flexible Lustre Find Utility (LFU) that can find files with arbitrary search criteria using different input streams, as well as be able to output lists of matching files and their attributes in multiple different text and binary formats, including data streams that are directly consumed by other applications.

## Filesystem Scanning Use Cases

The LFU scanning system will allow development of workflows for efficient data management. Ongoing file management is needed for features such as File Level Redundancy to resync stale data mirrors of recently written files, for automated file migration of old files from faster to slower storage pools, and for data import and export between external storage.

### File Level Redundancy Resync (FLR)

The current FLR mirrored files and upcoming Erasure Coded files (EC) operate in "delayed resync" mode, where files are written to the primary data components, but adding file redundancy requires an additional step to resynchronize the file to redundant components. This requires potentially processing all recently written files, but only files that have redundant file layouts.

## File Storage Tier Migration

To maximize efficient use of flash storage, less frequently used files can be migrated from flash to HDD storage. This requires locating the oldest files on the flash-based OSTs to remove them from those OSTs when they become full.

One option for balancing a multi-tier filesystem is to scan the flash OSTs to find the least-used files and then migrate them from to the HDD pool when the flash pool becomes full. However, this requires both scanning and migrating files under pressure to reduce flash usage more quickly than it is being consumed, likely when the flash pool is actively being written.

A better alternative is to monitor the Changelog for recently modified files to pre-mirror them from the flash pool to the HDD pool , using File Level Redundancy when they are first written, and then remove the flash mirror from the lease used files when the flash pool becomes full. This allows data movement when the OSTs may be less busy, and removing an FLR mirror can be done much more quickly than performing a large data copy. Also, this allows the flash mirror to be more fully utilized, since files can remain on the flash pool longer since they can be removed more quickly when it becomes full. In this configuration, the mirrored files may also need to be resynced if they are modified again after the mirror is created.

## Client-Side File Cache (PCC-RO)

The Persistent Client Cache Read Only feature stores a file mirror copy in a locally mounted filesystem to optimize access to frequently used files, and to reduce network traffic to the servers. This can be considered a form of external cache storage that is local to the client only. To manage the space used by the local cache directory, it also needs to be scanned periodically to find the least recently/frequently accessed files and remove that local copy.

## Trash Can Undelete (TCU)

The Trash Can Undelete feature requires scanning of the Trash Can folders to find the oldest deleted files to delete, to release space held by deleted files, and for new files to be written to the main filesystem. The list of files to purge from Trash could be generated by scanning the Trash Can directly, or it would be possible to track the TCU files in an index by deletion time to efficiently find the oldest files to delete when the free space becomes low.

## Filesystem Usage Reports

Efficiently generating reports for administrators is essential for management of large storage resources. LFU will be able to efficiently scan and aggregate results directly on the

server, regardless of whether the query is run on any server or client (with sufficient access permissions). This will be able to generate histograms of multiple file attributes, such as file ages, file and directory space usage, and reporting on a per-user, per-group, and per-project basis.

## Base Requirements

### Modular Architecture

The core infrastructure will have a modular/pluggable architecture for LFU that allows the initial development of a usable basic infrastructure, while facilitating LFU to be extended incrementally to handle other input sources, filters, and outputs in the future.

### Efficient Binary Data Exchange

The data interface between modules (Object Stream) that passes object FIDs and attributes between modules should be efficient without additional reparsing or data restructuring at each layer. There should be provisions in the Object Stream protocol to allow future expansion of the attributes being passed between modules without preventing interoperation with existing modules. High-performance structured binary formats such as FlatBuffers, ProtoBuf, or MessagePack should be investigated to provide efficient data transfer while allowing flexibility in the data transfer protocol rather than developing a custom binary data interface if allows sufficient performance.

### Core Module Types

There will be three types of modules initially that can be stacked together:

#### Input Scanner Modules

- Generate a list of objects (nameless FID-identified inodes) from a source (such as an MDT, OST, Changelog, or other index), optionally with attributes that are required by the dependent modules, or able to produce object attributes on demand.
- The Input Scanner modules should accept a mask of attributes that are required by the later filter and output modules to generate the Object Stream, to minimize the volume of data passed in the stream. It should be possible to distinguish between required and optional attributes, allowing attributes to be returned if readily available (e.g. the pathname from a namespace scan), but omitting the pathname if not (e.g. a file index that only stores the FID).
- The initial Input Scanner modules for locating files should be an ldiskfs filesystem scanner and Lustre Changelogs consumer.

- It should be possible to write the raw Object Stream to a file (via a "Raw Write" Output Format module) and then read from a raw Object Stream file as input (via a "Raw Read" Input Scanner module) to other Filter Rule or Output Format modules.

### Filter Rule Modules

- Consume one or more Object Streams and produce another object stream based on the object attributes and matching criteria.
- It should be possible to merge the output from multiple Input Scanner modules into a single object stream.
- It should be possible to stack multiple Filter Rule modules to refine the object stream returned by an Input Scanner module.
- The initial Filter Rule module should implement attribute filters similar to `find` and `lfs find`.
- Advanced operators such as "in list", "range" (min-max), "largest", "smallest", "histogram" for an attribute should optionally be supported by the protocol (even if not initially implemented for all attributes). This will allow efficient scanning and filtering when an index is available for that attribute, such as one of the UIDs in list, smallest atime (oldest files), or file size (largest files and directories).
- Operators such as "histogram" at the source allow efficient data aggregation for reporting, without the need to transport all of the attributes to the Output Format module. It should be possible to efficiently merge histograms from multiple sources.

### Output Format modules

- Consume an Object Stream and produce output in a different format. It should be possible to register multiple Output Format modules to an Object Stream, and to different levels of the Object Stream to provide different types of output.
- The initial Output Format module should provide basic text-based printing of FIDs, and an Output Format for efficiently generating pathnames for the FIDs, and optionally printing attributes with formatted text output like "`lfs find -printf`" (which could initially be used to generate a variety of different text-based output formats).

## Scanning and Filtering Performance

The ldiskfs Input Scanner Module should be able to scan and generate an Output Stream matching attribute search criteria at a rate of 1 million objects per second per MDT, assuming at least 1GiB/s read rate on the MDT device, exclusive of pathname generation.

Performance should scale in parallel across all MDTs and MDS nodes in a Lustre filesystem, assuming sufficient hardware resources. This puts an upper limit on a full namespace scan for maximum-sized and fully populated ldiskfs MDTs (4B objects) at approximately 1h, though it will typically be shorter for less-full MDTs with higher bandwidth.

### Use of External Databases

The efficient operation of LFU *should not depend* on an external database, and primarily scan the MDT or OST directly to identify objects of interest. Population and storage of an external database of sufficient size to store all filesystem metadata requires hardware equivalent to at least all MDTs and MDSes in the filesystem, which could better be utilized as MDTs and MDSes themselves (and in turn also make LFU and the filesystem proportionately faster, rather than stranding resources to provide the database service). Keeping an external database uptodate is a challenge for numerous reasons, and is best avoided.

## Future Improvements Beyond Initial Implementation

### Structured Output Format Modules

Beyond the initial functionality for plain text Output Format modules, other Output Format modules that provide standard structured data formats would be desirable, such as JSON, BSON (Binary JSON), Parquet (binary), etc. This would allow the LFU Object Stream to be efficiently used by a wide variety of tools.

Additionally, providing optimized Output Format modules plugins for direct ingest by RobinHood, PoliMor, GUFI and other consumers should be encouraged over having them use less efficient namespace scanning tools on the clients, which generate excessive load on the servers and network.

### Use of Internal Indexes

It may be desirable in some cases to maintain dynamic indexes for attributes (such as efficient atime scanning, subdirectory space usage, etc.). These indexes should preferably be handled as part of core MDS or OSS operations so that they are atomically updated with the operations (like Changelogs).

### Persistent Aggregates

In some cases, it may be useful to store aggregate/intermediate results (e.g. summary of directory usage statistics) in database shards in the filesystem itself, in order to use the

high-performance storage of the filesystem itself without needing external storage/servers, similar to the Grand Unified File Index (GUFI) [1] [2].

The Aggregates could be stored in upper-level directories should provide summaries of the namespace hierarchy where they are the root. Like Lustre `stats` files, the Aggregate file would include the minimum, maximum, count, and sum, (sumsquare?) for each of the standard attributes (e.g. size, blocks, atime, mtime, ctime, etc.) separately for both regular files and directories. If a subdirectory tree has common access permissions and ownership (e.g. a single user, group, project), it is also possible to provide per-user, per-group, and per-project summaries for regular users in those directories.

The benefit of Aggregates is that it allows efficiently summarizing results and pruning namespace searches based on matching or non-matching subdirectory trees. The drawback of storing and generating Aggregates is that they impose overhead to keep uptodate, and may be outdated due to file IO in subdirectories that have not been included in the Aggregate. This can be partly mitigated by having efficient (e.g. Changelog-driven) bottom-up updates after files and directories are modified. For example, it is sufficient to track the parent directory FID once for any number of files created, written, deleted in that directory to trigger reconstruction of that directory's Aggregate, and then roll up the changes to the parent Aggregates.

Rather than provide complex functionality in this area, it may be preferable to integrate the low-level scanning closely with GUFI, which already provides an ecosystem for this functionality.

## Server-Side Changes

### ldiskfs Device Input Scanner Module

The most efficient mechanism to generate an Object Stream for ldiskfs is by directly scanning the MDT or OST storage device, using deep knowledge of the backing filesystem and the libext2fs library to access the filesystem metadata. This follows the well-known design of similar tools, such as e2scan [1], and Lester, the Lustre Lister [2] and can scan at the read bandwidth of the underlying storage device. The low-level device scanner can run independently of the MDS and OSS service, though it requires read-only access to the underlying block devices.

### Inconsistencies in Scanning Results

One caveat with low-level device scanning is that the on-disk metadata may be temporarily inconsistent with the in-memory state for possibly tens of seconds due to a delay between

RPCs being processed by the MDS and those changes being committed to persistent storage, depending on server load and storage performance. This would only impact objects that are created, modified, or removed a few seconds before they are scanned, which would represent only a tiny fraction of objects.

Since the scan itself will take some non-zero time to complete, it cannot be "atomic" with respect to files and directories being created or removed during the scan, unless running on an idle filesystem (which may be the case when decommissioning an OST). Scanning a snapshot would also not see recent changes, so does not fundamentally improve the situation vs. running on a live filesystem. Most consumers are concerned with high-level aggregate results (how much space a user or directory is using, histogram of file age, migrating some number objects off an OST, resyncing stale files, etc.) for which an inconsistency of a few individual objects will not significantly alter the results.

### Changelog Input Scanner Module

Changelogs are generated in parallel on each MDT and provide an efficient mechanism to monitor ongoing filesystem modifications. The Changelog Input Scanner Module, together with the existing server- and client-side interfaces are sufficient for many purposes. The Changelog can be configured on a per-consumer basis to return records when specific filesystem *operations* are performed, such as creating new files or directories.

### Changelog Output Filter Module

There is currently no mechanism to filter Changelog events based on specific *attributes* (such as objects created by UID 1000, or files over 1TB in size). Implementing an Output Filter Module integrated with the Changelog Input Scanner Module on the MDS may be advantageous in some use cases to reduce the number of unnecessary Changelog records appearing in the Object Stream. In many cases, Changelog consumers are only interested in a specific subset of objects being modified (e.g. by user, or whether a redundant file layout is stale), so pre-filtering can reduce processing overhead significantly.

### Index Input Scanner Module

Certain types of scans, such as "find oldest files" can be significantly accelerated by maintaining and scanning an index directly to generate the Output Stream. In such cases, it is likely most efficient for the index to contain only the object FID, and possibly the index attribute so that the index can be maintained in order if that is necessary (vs. "buckets" of objects of similar values). The Object Stream would only contain the list of FIDs, unless other attributes are required, in which case the attributes can be populated from the objects directly.

### Oldest Access Time Index

For an "oldest files" index it may be sufficient to store object FIDs in index files, such as an `llog` file, named by the atime that references FIDs with a small range of atime values. Under normal operation, atime index files would be created with the "current time" and would roll over to new files once they contained enough FIDs (e.g. 1M) or if the atime was sufficiently different (e.g. 1h). FIDs would be deleted/cancelled from the current index file if the object is deleted, or if atime was changed beyond the range of the current file and inserted into a new index file. The atime index files would be deleted once they no longer contain any FIDs.

### Largest File Index

It may be desirable to have a "largest file" index to speed up locating and processing files that are consuming the most space. This should be indexing the *space* used by a file (`i_blocks`), rather than the offset of the last byte (`i_size`) since it is usually the capacity used by a file that requires attention. Since it is unlikely that there is a need to scan for the "smallest files", it may be sufficient to index only some fraction of files that are, say, using at least 1% of the total filesystem capacity, or similar.

## Binary Object Stream Format

There are a number of different structured data formats that could be used for the Object Stream. Most discussions about this indicate that while JSON is very flexible and easy to use, it universally provides the worst performance and space efficiency due to ASCII encoding and decoding of all values, and re-encoding the key names in each instance. Because the Object Stream will consist of (potentially) billions of identical objects (essentially on-disk inodes, with some flexible fields such as xattrs and filenames), it makes sense to encode the data structure as efficiently as possible. A binary format should be used instead of ASCII. While it would be possible to develop a custom protocol for this, it would be more flexible for integration into other toolchains to use a cross-platform data interchange format. Some investigation shows that FlatBuffers may be best suited to this task, due to efficient data representation using a pre-built schema rather than a flexible key-value structure for every object in the stream. The FlatBuffers format allows direct data accessing by field offset rather than having to parse and decode every object in the stream.

## OSD API Input Scanner Module

As an alternative to scanning the low-level device, it would be useful to interface directly with the running OSD device (`osd-ldiskfs`, `osd-zfs`, `osd-wbcfs`) to have the OSD perform the scanning itself. The ldiskfs and ZFS OSDs already implement a low-level

device scanner for OI Scrub and LFSCK [3], so it would be possible to generate an Object Stream directly from the OSD scanner for a userspace Filter Rule or Output Format module.

Using an OSD API Input Scanner module to generate an Object Stream has some advantages over low-level device scanning. The object metadata will be uptodate when the objects are scanned, since the scan would access the objects in memory instead of from the storage device. There would not be any need to allow the scanner to directly access the storage device, which improves security. While direct device access is reasonable for ldiskfs due to the relatively static metadata layout, it significantly more complex for ZFS, and incongruent with WBCFS. The userspace OSD scanning API would be consistent across different OSD backends, and would not need to have a *separate* scanner (though it would still need to be implemented for new OSDs). The OSC Scanning API would re-use the OSD OI Scrub scanner, so optimizations in this code would benefit both LFU and LFSCK (which may itself be restructured into an Object Stream consumer).

Some drawbacks of using the OSD scanner are that there is some additional overhead to process the objects in a consistent manner as part of the OSD code vs. reading them directly from storage in userspace. There may be additional overhead interfacing to the kernel to provide large volumes of structured data to userspace LFU filter modules, which might require at least basic attribute filtering to be implemented in the kernel to reduce the volume of data passed to userspace.

### Kernel API for Object Stream

The Linux kernel Circular Buffer Interface (`circ_buf.h`) provides a very efficient interface that could be used to pass the Object Stream from the kernel to a userspace Filter Rule module. This allows lockless zero-copy export of large volumes of data at near-memory speeds, essentially limited by how fast it can be populated in the kernel.

## Server Bulk RPC Filter Rule Module

It would be desirable to have a Filter Module that can export one or more Object Streams from the MDS or OSS to a client node, similar to Changelog access on the client. This would allow clients to perform efficient filesystem scans without having to traverse the filesystem namespace one object at a time and flood the dcache on the client. Combined with server-side authentication and result filtering to ensure the Object Stream contains only objects accessible by the user, this can provide very high-performance scanning results to users and applications.

# Client-Side Changes

## Lustre Namespace Input Scanner Module

As a starting point for API development and to provide broad functionality across different runtime environments, it makes sense to implement an Input Scanner module that generates an Output Stream by traversing the Lustre namespace by directory. This could be extracted from LU-17814 "'`lfs find`' to scan with multiple threads" to generate the Object Stream in userspace initially using the existing `ioctl(LL_IOC_MDC_GETINFO_V2)` interface to fetch individual file attributes. This would also be useful for client-side workloads using LFU interfaces before full LFU implementation.

## "`lfs find`" Filter Rule and Output Format Modules

Implementing "`lfs find`" as Filter Rule and Output Format modules on top of the Lustre Namespace Input Scanner Module would provide early feedback on API and data structure choices and have the benefit of converting the existing infrastructure over to using the new Object Stream APIs rather than leaving "`lfs find`" as future technical debt.

## POSIX Input Scanner Module

It may be desirable to also implement an Input Scanner Module that performs a traditional POSIX directory traversal for non-Lustre filesystems to generate an Output Stream. This would be useful for processing non-Lustre storage to integrate into the LFU toolchain for various functions such as Client-Side File Cache (PCC-RO) and Trash Can Undelete (TCU). This could likely be implemented as part of the Lustre Namespace Input Scanner module using `statx()` calls to fetch file attributes from the kernel rather than implementing a duplicate parallel namespace scanner module.

## Client Bulk RPC Filter Rule Module

To have highly efficient scanning on the client, it should be possible to attach a Client LNet Filter Rule Module to the Server LNet Filter Rule Module so that the Object Stream is transferred via bulk RDMA directly to the client. Subject to directory access permissions (taking into account regular POSIX UID/GID/ACL access permissions, filesets, nodemaps, etc.) it should be possible to efficiently fetch and process an Object Stream on a client node, similar to a Lustre Changelog, using the same Kernel API for Object Stream as the OSD to ensure uniformity of access in both cases.

Once the Client LNet Filter Rule Module is available, it should be possible for "`lfs find`" to export search requests directly to multiple servers in parallel rather than scanning the namespace locally. This could be used initially by administrators on a client rather than running it on the server, and later by regular users to efficiently perform directory scans.

That would both improve performance and reduce server overhead by reducing the number of files accessed by the client(s). Server offload of namespace scanning has successfully been used to improve IO500 `find` performance.

## Feature Compatibility and Interoperability

The LFU interface will be new and not itself provide compatibility with older clients or servers. However, compatibility will depend on how LFU is being used.

### Usage on Old Servers

While the OSD API Input Scanner Module would not be available on an old server, the ldiskfs Device Input Scanner Module would be available to directly scan the underlying MDT or OST storage without having any code changes in the Lustre kernel modules. It can generate an Object Stream by directly scanning the ldiskfs filesystem.

### Usage on Old Clients

It is possible to utilize the Lustre Namespace Input Scanner Module (directory scanner) to generate an Object Stream on an old client because it is using a long-standing API for "`lfs find`" to scan directories in the filesystem and access object attributes from the client. While this would not be faster than running the old "`lfs find`" directly, it would allow old clients to start using LFU APIs to integrate with related workflows.

### Client and Server Interoperation

If the server does not provide the Server LNet Filter Rule Module, or the client does not provide the Client LNet Filter Rule Module, then it would not be possible for clients to use LFU to execute the Input Scanner operation to the server. The availability of this server-side scanning functionality would be negotiated with a new `OBD_CONNECT2_LFU` feature flag at client connection time. The LFU Object Stream should be implemented with flexibility to provide interoperation across releases, with protocol flags to identify feature availability and usage, rather than version numbering. This is one of the reasons to use an extensible binary data format that can add features as needed, rather than monolithic data structures.

## Testing New Functionality

Regression test cases should be implemented to cover various aspects of the LFU interface and module implementation:

- Scanning of MDT and OST devices to ensure full reporting of files and objects stored therein

- Scanning of a directory tree to ensure full reporting of files and directories therein
- Object Stream encoding, decoding, and structure to ensure accurate content and compatibility between releases
- Filter Rules tests for each module to ensure correct operation
    - Existing "`lfs find`" test cases will already cover many of the filter rules for this module
    - Aggregate Filter Rule testing should confirm accuracy of the min, max, count, sum, and histogram values
- Output Format tests for each module to ensure structure (e.g. JSON) and accurate Object Stream rendering
    - Pathname generation from FIDs should be verified

## References

[1] D. Manno and J. Lee, "GUFI: Grand Unified File Index," Los Alamos National Laboratory, 2022. [Online]. Available: https://sc22.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap199.html.

[2] "GUFI: Grand Unified File Index Github Repository," [Online]. Available: https://github.com/mar-file-system/gufi.

[3] A. Dilger, "e2scan Utility," Cluster Filesystems, 2007. [Online]. Available: https://review.whamcloud.com/plugins/gitiles/tools/e2fsprogs/+/refs/heads/v1.42.13.wc6-lustre/e2scan/.

[4] D. Dillow, "Lester, the Lustre Lister," Oak Ridge National Laboratory, 2013. [Online]. Available: https://github.com/ORNL-TechInt/lester.

[5] F. Yong, "LFSCK Phase 1 - OI Scrub Solution Architecture," Intel, 2012. [Online]. Available: https://wiki.lustre.org/LFSCK_Phase_1_-_OI_Scrub_Solution_Architecture.

[6] S. Buisson, "Limit capabilities of local admin," 05 2023. [Online]. Available: https://jira.whamcloud.com/browse/LU-16524.

[7] B. Faccini, "Special File/Dir to Represent DAOS Containers," 05 2019. [Online]. Available: https://jira.whamcloud.com/browse/LU-19884.

[8] D. Wang, "Allow Cross-MDT for All Metadata Operations," 06 2015. [Online]. Available: https://jira.whamcloud.com/browse/LU-3537.